Parte A - Setup & Arquitetura (Sprint 1) Respostas das questões

Tema 1 – O que é Git e por que usar

1- O que é controle de versão e qual problema ele resolve numa equipe que programa junto?

É uma forma de que várias pessoas registrem dados e organizar as mudanças sem que os arquivos anteriores sejam modificados. Ele resolve o problema de exclusão de arquivos e sobreposições das informações.

2-Qual a diferença entre Git (a ferramenta) e GitHub (o site/serviço)?

Git é a ferramenta de controle de versão onde várias pessoas podem criar controles de versão e código-fonte.

Já o GitHub é uma plataforma online para organizar os projetos em repositórios para melhor organização.

3-O que é um "repositório" (repository) e o que é um "commit"?

Repositórios são como uma pasta onde ficam salvos os projetos.

Já o Commit é quando as informações são salvas por etapa, fazendo com que o usuário possa rever e modificar versões anteriores do mesmo projeto.

Tema 2 – Fluxo de Branches (Main / develop / feature)

1-O que é uma "branch" (ramificação) dentro de um repositório Git?

Uma branch no Git é como uma cópia do projeto onde você pode fazer alterações sem mexer no projeto principal.

2- Por que times profissionais separam a branch main (estável) da branch develop (em construção)?

Para proteger a versão estável, para não modificar.

3- O que significa criar uma branch chamada feature/nome-da-tarefa? Dê um exemplo de nome pra uma tarefa do nosso projeto.

Significa criar uma branch separada para desenvolver uma tarefa específica, sem mexer diretamente na develop.

Exemplo: feature/tela-de-login

Se a tarefa do projeto fosse criar a tela de login, esse seria um nome adequado para a branch

Tema 3 — Conventional Commits

1- O que é o padrão "Conventional Commits" e qual problema ele resolve?

Conventional Commits é um padrão para escrever mensagens de commit de forma organizada, para que qualquer pessoa consiga entender o histórico do projeto sem precisar abrir cada alteração

2-Qual a diferença entre um commit feat, um fix, um docs e um test?

Os tipos de commit servem para mostrar o que foi feito no projeto. O feat é usado quando criamos alguma coisa nova, o fix quando corrigimos algum erro, o docs quando mudamos alguma documentação e o test quando fazemos ou alteramos testes para ver se o código está funcionando. Assim, fica mais fácil entender o que mudou em cada parte do projeto.

3-Escreva um exemplo de mensagem de commit no padrão para "criar a tela inicial do jogo de dado".

Como foi criada uma nova tela para o jogo de dado, usamos o feat, porque ele serve para adicionar uma nova funcionalidade ao projeto. Então, a mensagem do commit ficaria: feat: criar tela inicial do jogo de dado.

Tema 4- Pull Request e Code Review

1. O que é um Pull Request (PR) e para que ele serve, além de só "juntar código"?

Além de “juntar os códigos” Pull Request é uma forma de pedir para que outras pessoas confiram os dados adicionados por um membro do grupo, dê sugestões e corrija caso for necessário antes que esses dados sejam lançados ao projeto principal.

2. O que é Code Review e por que revisar o código de um colega antes do merge é uma prática de mercado?

Apresentação

O nosso grupo ficou responsável pela **parte matemática do site**. A nossa ideia foi mostrar, através de cálculos e de uma simulação, como funciona a vantagem matemática da casa em um jogo de apostas.

Para isso, a gente trabalhou principalmente com **valor esperado** e **simulação Monte Carlo**.

Valor esperado

Primeiro, temos o valor esperado, que basicamente mostra qual seria o resultado médio de uma aposta se ela fosse repetida muitas vezes.

No nosso exemplo, o jogador tem **49% de chance de ganhar** e **51% de chance de perder**.

Então, se ele apostar 100 PRIMEs, fazemos:

E[X] = (0,49 × 100) + (0,51 × -100)

Isso dá:

E[X] = -2 PRIMEs.

Ou seja, a cada 100 PRIMEs apostados, a perda média esperada é de 2 PRIMEs.

É daí que vem a **margem de 2% da casa**.

Mas é importante deixar claro que isso não significa que uma pessoa vai necessariamente perder 2 PRIMEs em cada aposta. Ela pode ganhar várias vezes seguidas ou perder várias vezes. Esses 2 PRIMEs representam uma **média esperada quando fazemos muitas apostas**.

Simulação Monte Carlo

Para mostrar isso na prática, a gente criou uma simulação Monte Carlo.

Em vez de fazer somente a conta no papel, o computador realiza milhares de rodadas aleatórias.

Por exemplo:

> Rodada 1: ganhou.
> Rodada 2: perdeu.
> Rodada 3: perdeu.
> Rodada 4: ganhou.

E assim por diante.

A cada rodada, o programa atualiza o saldo e guarda o resultado.

Depois de milhares de rodadas, conseguimos comparar o resultado que realmente aconteceu na simulação com aquilo que a matemática previa.

Parte interativa

Também deixamos o simulador interativo.

A pessoa pode escolher **quantos PRIMEs virtuais quer colocar em cada rodada**, quantas rodadas quer fazer, qual será a chance de vitória e qual será o saldo virtual inicial.

Por exemplo, se colocarmos **500 PRIMEs por rodada**, com 49% de chance de vitória:

E[X] = (0,49 × 500) + (0,51 × -500)

O resultado é **-10 PRIMEs por rodada**.

Então, quanto maior for o valor colocado em cada rodada, maior será o valor absoluto da perda ou do ganho esperado, embora a porcentagem continue sendo a mesma.

Muitas rodadas

Se tivermos uma perda esperada de 10 PRIMEs por rodada e fizermos 10.000 rodadas:

10 × 10.000 = 100.000 PRIMEs.

Então, matematicamente, esperamos uma perda de aproximadamente 100.000 PRIMEs nesse cenário.

Mas o resultado real não precisa ser exatamente esse, porque a simulação utiliza números aleatórios.

Por isso, podemos terminar um pouco acima ou um pouco abaixo desse valor.

O gráfico

Para facilitar a visualização, o site também cria um gráfico mostrando o saldo ao longo das rodadas.

No eixo horizontal temos o **número de rodadas**, e no eixo vertical temos o **saldo virtual**.

Assim conseguimos visualizar as oscilações causadas pela aleatoriedade e observar o comportamento do saldo ao longo da simulação.

A questão da casa "justa"

Depois dessa parte, analisamos a pergunta principal do projeto: **seria possível criar uma casa em que o jogador não perdesse seu capital principal e, ainda assim, existisse uma forma de gerar os prêmios?**

No modelo tradicional, existe uma vantagem matemática para a casa. Então, quando o jogo é repetido muitas vezes, o resultado esperado favorece a casa.

Já a proposta do projeto, chamada de **No-Loss Pool**, funciona de outra maneira.

A ideia é que o capital principal dos jogadores fique separado em um pool ou contrato de custódia, e que os prêmios sejam pagos a partir do rendimento gerado por esse capital.

Então, em vez de o dinheiro de um jogador perdido diretamente virar o prêmio de outro, a proposta seria utilizar o rendimento do pool.

Conclusão

Então, resumindo, o nosso trabalho teve três partes principais:

Primeiro, usamos o valor esperado para demonstrar matematicamente a margem da casa.

Segundo, criamos a simulação Monte Carlo para testar essa ideia com milhares de rodadas.

E terceiro, deixamos o simulador interativo, para que seja possível mudar os valores e observar como a quantidade de PRIMEs, a probabilidade e o número de rodadas alteram o resultado esperado.

Com isso, a gente conseguiu transformar a parte matemática do projeto em algo que pode ser **calculado, simulado e visualizado no próprio site**.